

Where Are They Now?

Calibrated Tracking of Unseen Opponents in *Counter-Strike 2* from Team-Level Information

Akash Khanikor

Washington University in St. Louis

khanikor@wustl.edu

September 2026

Abstract

In tactical shooters such as *Counter-Strike 2* (CS2), most of a round is played against opponents who cannot be seen. We study the problem of estimating, at every moment of a recorded match, a probability distribution over the location of each opponent who has dropped off a team’s radar, using only information that team possessed at the time. We present CSPREDICT, a factored Bayes filter over a navigation grid learned from player positions (1,999 cells grouped into the map’s 23 named callouts). Its components are learned from 82 recorded matches on the map *de_mirage*: a logistic spotting model, fitted on about 27 million observer–enemy pairs, that turns “a teammate looked here and saw nobody” into evidence; a particle filter whose particles replay recorded trajectories from similar situations, blended with a grid Markov model and a population prior; learned molotov-avoidance profiles; a coordination correction based on about 134,000 recorded snapshots of whole teams; and a post-hoc calibration that depends on how long ago the opponent was last seen. On 16 held-out professional matches (102,486 evaluation moments), CSPREDICT names the correct callout 48.1% of the time (72.7% within its top three), against 42.2% for the common “they are where I last saw them” heuristic and 21.3% for a time-conditioned population prior. Its callout log-loss is 1.665 nats, against 7.196 and 2.607, and its stated probabilities are well calibrated (expected calibration error 0.0012). Ablations attribute most of the gain to replayed trajectories and to negative information, while several intuitive features, such as the weapon an opponent was last seen holding, give no measurable improvement.

Keywords: Bayesian filtering, particle filters, partial observability, negative information, calibration, proper scoring rules, esports analytics.

1 Introduction

Counter-Strike 2 (CS2) is a five-versus-five tactical shooter. Each player sees only what lies in their own line of sight, plus a shared minimap, the *radar*, that marks any enemy a teammate currently sees. Most of a round is therefore played against opponents whose positions are unknown. Strong players and coaches reason about those positions constantly (“they were last seen on A ramp eight seconds ago, nobody has come through Palace, so they are probably still near the site”), and much of post-match review consists of reconstructing and critiquing exactly this reasoning.

This paper asks whether that reasoning can be made quantitative. Given only what one team knew at a moment of a recorded match, we estimate, for each opponent who has dropped off the radar, a probability distribution over where that opponent is now (Figure 1). The distributions are meant for demo review. A coach can scrub through a round and see which areas were plausibly occupied, how the uncertainty grew as

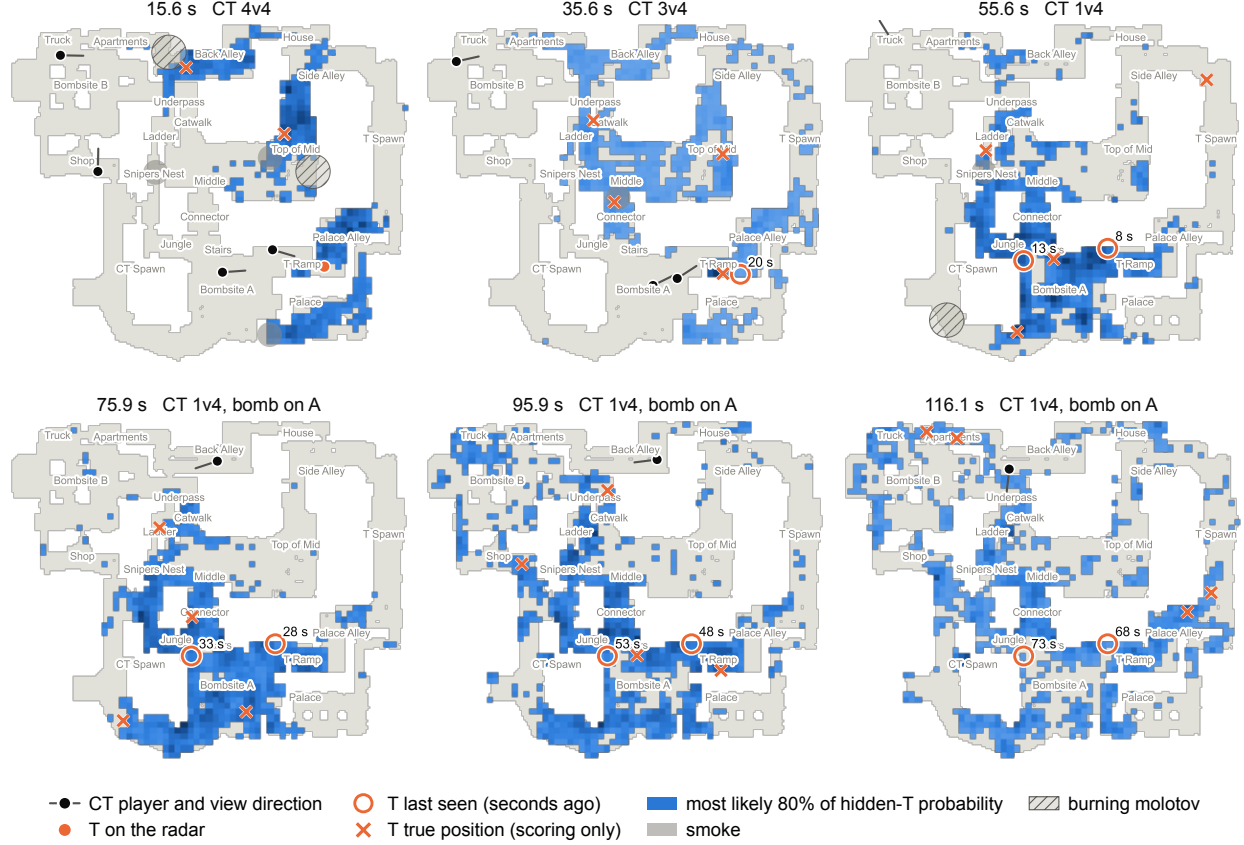


Figure 1: Six moments of a held-out professional round (EAC vs. lilmix, round 9) from the Counter-Terrorist (CT) side, which ends in a 1-versus-4 with the bomb planted on A. Blue cells hold the most likely 80% of the probability for the hidden Terrorists (Ts), with darker cells more likely. Rings mark where each T was last seen and how long ago. The orange crosses show where the Ts really were; they are drawn for review only and are never seen by the model. Grey discs are smokes and hatched discs burning molotovs.

the team lost track of an opponent, and whether a player’s decisions were consistent with the information available at the time.

Four features make the problem harder than it first appears.

- (i) **Multimodal uncertainty.** Maps are networks of corridors, so an opponent who may have gone either way is in one corridor or the other, not in the wall between them. A single Gaussian estimate, as in the Kalman filter, describes this poorly.
- (ii) **Negative information.** Most of the evidence is the *absence* of a sighting. A teammate who looks down a corridor and sees nobody makes that corridor unlikely, but how unlikely depends on sight lines, distance, the teammate’s view direction, smokes and flashes.
- (iii) **Situation-dependent behaviour.** How far an opponent travels depends on how long ago they were seen, whether the bomb is planted, the team’s economy and the phase of the round.
- (iv) **Coordination.** The five opponents act as a team. Tracking each one independently ignores that teams stack a bomb site, send a lurker, or hold a fixed set-up.

In addition, the output must be *calibrated* (when it says 70%, the opponent should be there about 70% of the time), and the evaluation must use scores that reward honest probabilities rather than confident guesses.

Our system, CSPREDICT, addresses these points with a factored Bayes filter: one belief per opponent over a grid of about two thousand cells, updated every 0.25 s. Every component is learned from recorded matches.

The map’s geometry, which we could not obtain, is learned from where players stand. Sight lines and a spotting model are learned from the game’s own record of who saw whom. The motion model replays real recorded trajectories from similar situations. A final correction accounts for team coordination and calibrates the output. The closest prior work, by [Hladky and Bulitko \(2008\)](#), tracked opponents in *Counter-Strike: Source* with 190 LAN games on one map. We revisit the problem with modern professional data, full probability distributions and proper scoring rules.

Contributions.

- A formulation of opponent tracking from team-level information in CS2, with an information state restricted to what the team could know (up to one approximation, the opponents’ economy class, which an estimate from public information can replace at no measurable cost), and an evaluation protocol based on proper scoring rules, round-level bootstrap intervals and paired comparisons (Sections 3 and 6).
- A complete filter learned from demo files alone. It needs no map geometry and combines a logistic spotting model for negative information with a trajectory-library particle filter conditioned on the time since the last sighting (Section 5).
- A coordination correction that sums over every pairing between tracked opponents and recorded team snapshots, and a calibration that depends on the forecast horizon. We show why the former must wait for first contact and why the latter must depend on the horizon (Sections 5.7, 5.8 and 8).
- An empirical study on 16 held-out professional matches, with ablations of every component, features that did *not* help, and a comparison with a general-purpose model queried through text descriptions (Section 7).

The intended use is post-match analysis. Used live during a match, a tool like this would give players information they are not entitled to, which competitive platforms forbid (Section 9).

2 Background and Related Work

2.1 Bayesian filtering

A *Bayes filter* tracks a hidden state x_t , here an opponent’s position, from a sequence of observations $z_{1:t}$ ([Thrun et al., 2005](#)). It maintains a *belief* $b_t(x) = P(x_t = x \mid z_{1:t})$ and alternates two steps:

$$\text{predict: } \bar{b}_t(x) = \sum_{x'} P(x_t = x \mid x_{t-1} = x') b_{t-1}(x'), \quad (1)$$

$$\text{update: } b_t(x) = \frac{P(z_t \mid x_t = x) \bar{b}_t(x)}{\sum_{x''} P(z_t \mid x_t = x'') \bar{b}_t(x'')}. \quad (2)$$

The prediction spreads the belief according to a *motion model*; the update is Bayes’ rule with an *observation model* $P(z_t \mid x_t)$. When the dynamics are linear and all noise is Gaussian, the belief stays Gaussian and the filter reduces to the Kalman filter ([Kalman, 1960](#)). For multimodal beliefs one either discretizes the state (a *grid* or histogram filter) or represents the belief by weighted samples (a *particle filter*; [Gordon et al., 1993](#); [Kitagawa, 1996](#); [Doucet et al., 2001](#)). Particle filters periodically *resample*, duplicating heavy particles and dropping light ones, when the effective sample size $1/\sum_m w_m^2$ of the normalized weights falls too low ([Kong et al., 1994](#)).

Updating on the *failure* to detect a target, known as negative information, is classical in search theory ([Stone, 1975](#)). If a sensor would detect a target at x with probability $\pi(x)$ and reports nothing, the likelihood of that report is $1 - \pi(x)$, which removes probability from well-covered places. Figure 2 illustrates one predict–update cycle of this kind.

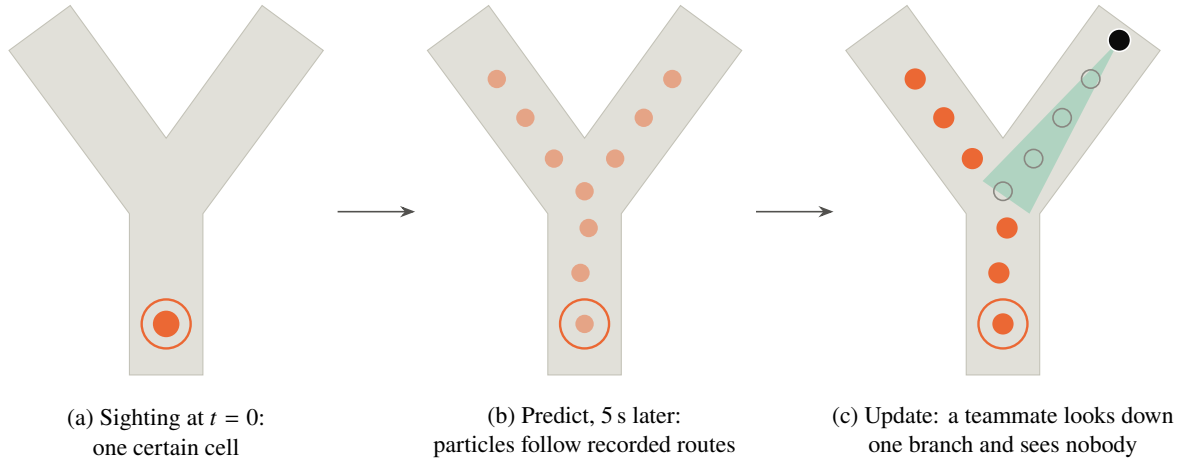


Figure 2: One predict–update cycle, schematically. An enemy is seen at the foot of a forked corridor (orange ring). As time passes, the belief (orange dots, each a particle carrying probability) spreads along the routes players really take. A teammate (black dot) then looks down the right branch (green wedge) and reports nothing. The particles in view lose their weight (grey circles), and after renormalization the remaining probability, in the left branch and the corridor, grows.

Our particle motion model follows *analog forecasting* (Lorenz, 1969). Instead of a parametric motion model, each particle follows a recorded trajectory that passed through a situation similar to the current one. This is a kernel-weighted nearest-neighbour method (Cover and Hart, 1967) applied to whole futures rather than to single labels.

2.2 Opponent modelling in games

The problem of locating hidden opponents appears across game genres. Isla (2006) describes *occupancy maps*: a probability grid that diffuses over a level and is erased wherever the agent can see, which is our DIFFUSE baseline. Bererton (2004) proposed particle filters for game AI, and Weber et al. (2011) learned particle models from expert *StarCraft* replays. Synnaeve et al. (2018) and Jeong et al. (2020) predict hidden *StarCraft* units with neural encoder–decoders and generative adversarial networks. Tastan et al. (2012) combined learned motion models with particle filters to intercept opponents in *Unreal Tournament*. OpenAI Five handled fog of war in *Dota 2* partly by reusing the last observation of unseen heroes (Berner et al., 2019), which is our LASTSEEN baseline. See Yannakakis and Togelius (2018) for a broader survey.

For Counter-Strike specifically, Hladky and Bulitko (2008) and Hladky (2009) built per-opponent hidden semi-Markov models and particle filters for *Counter-Strike: Source*. Their observation model used negative information from the friendly team’s field of view. Trained on 140 LAN games on one map and evaluated on 50, their models matched or exceeded human experts at predicting where Terrorists were. They produced point predictions evaluated by path distance, modelled opponents independently, and did not condition on game state such as economy. Recent work on CS2 approaches the question from other angles. X-Ego (Wang et al., 2025) predicts which callouts contain opponents from first-person video with the minimap masked out; we use its recordings as a second data source. MLMove (Durst et al., 2024) learns professional movement for bots from observations of all ten players, and Xenopoulos et al. (2020) and the ESTA dataset (Xenopoulos and Silva, 2022) support win-probability and player-valuation models. None of these produces a calibrated belief over hidden opponents from team-level information.

2.3 Evaluating probabilistic forecasts

A scoring rule is *proper* if a forecaster minimizes its expected score by reporting their true beliefs (Gneiting and Raftery, 2007). The logarithmic score (Good, 1952) and the Brier score (Brier, 1950) are proper. Rules such as “probability within 300 units of the truth” are not, and they reward over-confident point guesses (Section B.1). A forecaster is *calibrated* if events it assigns probability p happen with frequency p (DeGroot and Fienberg, 1983). Calibration is usually checked with reliability diagrams and summarized by the expected calibration error (Naeini et al., 2015), and it can be repaired after the fact by monotone maps such as Platt scaling (Platt, 2000) or temperature scaling (Guo et al., 2017). For uncertainty intervals we use the bootstrap (Efron, 1979; Efron and Tibshirani, 1993), resampling whole rounds as clusters because moments within a round are strongly dependent (Field and Welsh, 2007).

3 Problem Formulation

Map and time. We discretize the walkable area of the map into cells $i \in \mathcal{V} = \{1, \dots, N\}$ with $N = 1,999$ (Section 5.2). A labelling $c : \mathcal{V} \rightarrow C$ assigns every cell to one of $|C| = 23$ *callouts*, the named regions players use to communicate (*Jungle, Palace, Bombsite A, \dots*). Time advances in steps of $\Delta = 0.25$ s from the end of the round’s freeze time. Positions are in game units; for scale, a standing player is 72 units tall and 32 wide, and running speeds are roughly 200–250 units/s.

Information state. Fix a round and a *friendly* side; the other side’s players are the *enemies* $e \in \mathcal{E}$, $|\mathcal{E}| \leq 5$. At step t the friendly team’s information $\mathcal{I}_t$ consists of:

- the position, yaw and pitch of every friendly player, and whether they are alive or blinded by a flashbang;
- for every enemy, whether it is on the radar and, if so, its cell $y_t^e \in \mathcal{V}$ (otherwise $y_t^e = \emptyset$) and the weapon in its hands;
- active smokes and burning molotovs or incendiaries;
- the kill feed (who killed whom, whether through a wall or smoke) and where friendly victims died;
- which enemies are alive (the scoreboard), the enemy team’s economy class at the start of the round (see the assumptions below), the round clock, and the bomb phase $\phi_t \in \{0, 1, 2\}$ (not planted, planted on A, planted on B).

The demo file also records every enemy’s true position x_t^e . These positions are withheld from the model and used only for scoring.

Task. For each alive enemy not on the radar, compute

$$b_t^e(i) = P(x_t^e = i \mid \mathcal{I}_{1:t}), \quad q_t^e(k) = \sum_{i: c(i)=k} b_t^e(i), \quad (3)$$

the cell belief and the implied *callout belief*. We denote by τ_t^e the seconds since enemy e was last on the radar ($\tau = \infty$ if never this round). Our evaluation targets the mid- and late-round case, meaning moments when an enemy is off the radar *but was seen earlier in the round*. Before first contact the best available answer is essentially “where that side usually goes,” which a population prior already provides; those moments are reported separately.

Assumptions. We assume, as the game’s interface makes largely true, that the team can tell which enemy it is seeing on the radar, and that burning molotovs are known to both teams: the thrower saw where it landed, and the flames are visible and audible. One input of the model evaluated in this paper is only approximately

part of the team’s information: the opponents’ economy class (eco, force buy or full buy) is computed from their actual equipment values when freeze time ends. A real team cannot see those values, but it can infer the class from public information, and doing so changes nothing measurable. An estimate from the round history and the guns the team has seen, on the radar or in the kill feed (Section 5.1), names the right class for 77% of validation rounds when freeze time ends, and at 92% of the validation moments we evaluate, by when guns have been seen. Using it instead of the true class changes callout log-loss by -0.001 (-0.008 to $+0.007$) on validation, averaged over two random seeds of the particle filter, and by -0.002 (-0.008 to $+0.004$) on test with the calibration refitted; top-1 accuracy changes by $+0.000$ (-0.002 to $+0.002$) and $+0.001$ (-0.001 to $+0.004$). Rerunning the particle filter with another random seed moves both metrics as much. The released code uses the estimate; the numbers reported in this paper use the true class. We do not use sound (footsteps, gunfire), by design; Section 9 discusses this choice.

4 Data

We use two sources of recorded CS2 matches on the map *de_mirage* (Table 1). **HLTV (professional)**: 71 maps, one from each of 71 series, downloaded manually from HLTV.org (HLTV.org, 2026) in September 2026. The events include CCT, ESEA, ESL Challenger League, iBuyPower Masters and Stake Pulse, so most teams are tier two or three, with some tier one. **FACEIT (development)**: the 46 Mirage matches of the X-Ego-CS dataset (Wang et al., 2025), played between players ranked in the top 100 on the FACEIT platform, with the dataset’s own train/validation/test split. The FACEIT matches were recorded on an earlier game patch (14094 against 14181–14185), but the Mirage layout is unchanged: 99.9% of professional positions fall on cells already mapped from the FACEIT data.

Demos record the game state at 64 ticks per second. We parse them with demoparser2 and awpy (LaihoE, 2026; Xenopoulos et al., 2026), keep every eighth tick (8 Hz), and run the filter every 16 ticks (0.25 s). For each player we keep position, view angles, alive status, equipment value, the active weapon, the game’s place name for the current position, and the game’s own radar state: a per-player *spotted* flag and a *spotted-by* list of the opponents who currently see that player. Events include kills (with wallbang and through-smoke flags), bomb plants, smokes, infernos and flashbangs.

Four data problems had to be solved along the way, and each is documented because it changed results. (1) awpy’s download server for map geometry, navigation meshes and collision triangles returned errors, so the geometry had to be learned from player positions (Section 5.2). (2) Without that geometry, awpy labels every bomb plant as bombsite B; we relabel each plant with the nearer bombsite, whose centre is the mean position of players standing where the game’s place name is Bombsite A or Bombsite B. (3) A parsing bug cut 15% of smokes 4 s short. (4) 55 of the 71 HLTV demos contain no `player_blind` events, so flashed players looked able to spot. We reconstruct flashes from each player’s networked flash duration, which jumps to exactly the event’s blind duration on the tick of the flash; on demos that contain both, the reconstruction reproduces every event that names a player.

Splits are made by whole maps, so no round of a test map is ever seen in training. HLTV maps are assigned 70/10/20% to training, validation and test by a stable hash of the demo identifier, which gives 47/8/16 maps. Models are trained on the 82 pooled training maps (47 HLTV and 35 FACEIT), all settings are chosen on validation maps, and the 16 HLTV test maps are used only to report results.

5 Method

Figure 3 gives an overview. Offline, we fit a navigation grid, a spotting model, two motion models, molotov profiles and a library of team snapshots on the training maps, and a calibration map on the validation maps. Online, the filter reconstructs what the team knew and runs a predict–update step for every hidden enemy

Table 1: Data. Each map is one game of a series; the two teams swap sides at half time, and a match on Mirage is up to 24 rounds plus overtime. Evaluation moments are (step, hidden enemy) pairs with the enemy seen earlier in the round, pooled over both teams’ viewpoints.

	HLTV (professional)	FACEIT (X-Ego-CS)
Maps: train / validation / test	47 / 8 / 16 (71)	35 / 5 / 6 (46)
Rounds: train / validation / test	1,030 / 171 / 329 (1,530)	793 / 107 / 132 (1,032)
Players	tier-2/3 professionals, some tier 1	top-100 FACEIT players
Game patch	14181–14185	14094
Split	stable hash of the demo id	provided with the dataset
Evaluation moments	63,906 (validation), 102,486 (test)	48,876 (test)

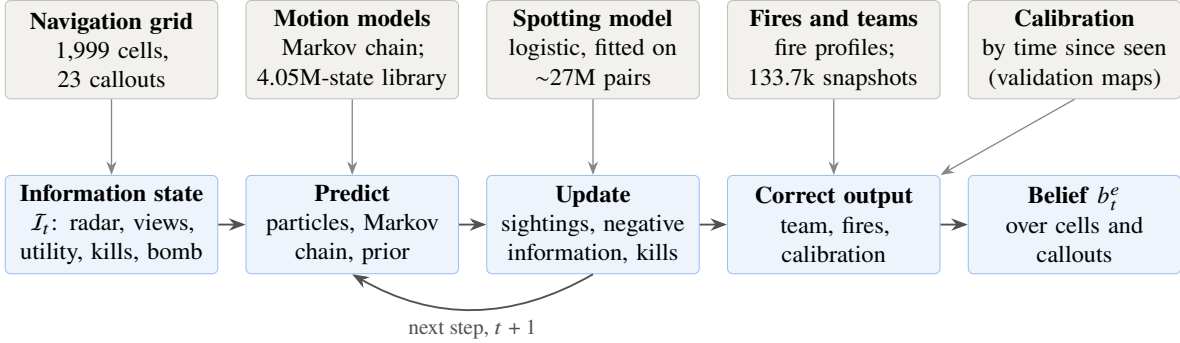


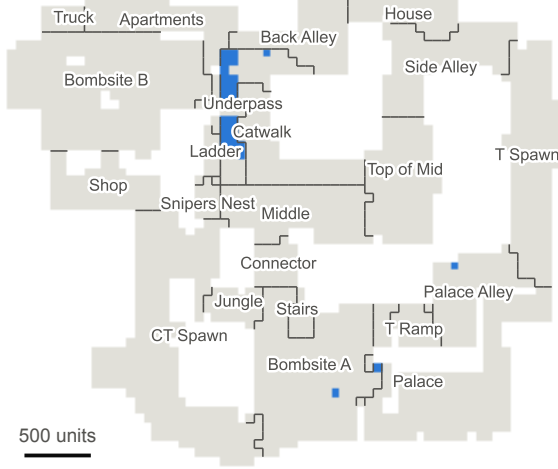
Figure 3: System overview. Grey boxes are fitted once; blue boxes run at every filter step. Grey arrows show the main fitted component each online stage uses; the fire profiles also reshape the grid chain’s predictions. The output corrections reshape only the reported belief and never feed back into the recursion.

every 0.25 s, followed by output-only corrections. Algorithm 1 summarizes one step, and Section A lists every constant.

5.1 Reconstructing the information state

An enemy is *on the radar* during a step if its spotted flag is set at either of the step’s two 8 Hz samples, and its sighting cell is the most recent one. A friendly player can spot only while alive and not blinded; a flash counts as blinding when it lasts more than 0.3 s, from the flash until its duration expires. Smokes block sight from 1.0 s after they burst until 1.5 s before they expire. We measured both offsets from spotting rates through smokes in the training demos, since a smoke needs time to bloom and thins out before it disappears. Molotovs count as burning from 0.5 s after they burst, when the flames have spread, until they burn out. A kill by an enemy is attached to the first step at or after it, together with the victim’s position and whether it went through a wall or a smoke. The enemy team’s economy is summarized by its mean equipment value when freeze time ends: an *eco* below 1,500, a *force buy* up to 3,800, and a *full buy* above that. A team cannot see its opponents’ equipment, so the released code estimates the class from public information instead (Section 3). The prior is how often each class was bought in the training rounds in the same economic situation, smoothed with 3 rounds at the overall frequencies. The situations are the pistol round, overtime, round 2 by the pistol-round result, round 3 after a loss in round 2 (again by the pistol-round result), any other round after a win, and any other round after a loss by its loss-bonus count (one up per round lost in the half, one down per round won, at most 4). Each enemy seen so far multiplies the prior by the probability, under each class, of the dearest gun that enemy has shown on the radar or in the kill feed (pistol, SMG, rifle or sniper), raised to the power 0.7 because teammates buy together. Recorded rounds keep their true class, and the economy factors of the matching kernels below (0.7 per class of difference) are then averaged over the estimated distribution of the

(a) 1,999 learned cells, 23 callouts



(b) Where Ts usually are 30–35 s into a round

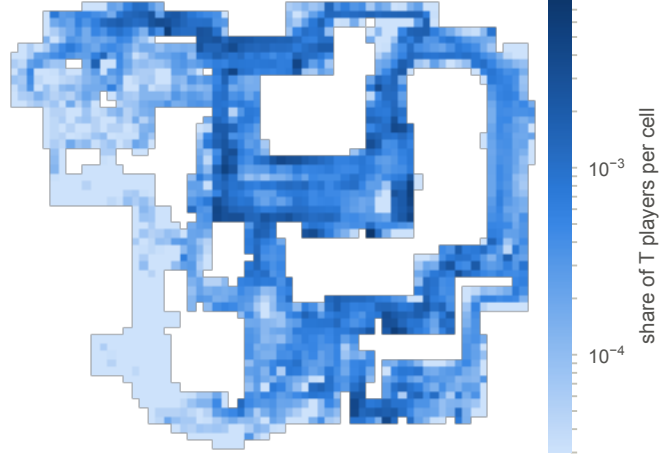


Figure 4: (a) The navigation grid learned from player positions. Lines separate callouts (on the lower floor), and blue marks the 27 map columns that hold two floors, mostly Catwalk above Underpass. (b) The population prior used as a baseline and as 10% of the ensemble: where Terrorists stand 30–35 s into a round before the bomb is planted, pooled over the training maps (log scale).

opponents' class.

5.2 A navigation grid learned from player positions

Without map geometry, we learn the walkable space from where players have stood (Figure 4a). The map is cut into columns of 64×64 units. Within a column, the recorded heights are split into floors wherever consecutive heights differ by more than 110 units (a jump reaches about 66), and every floor with at least one second of recorded presence becomes a cell. A column can hold several floors, as where Underpass runs beneath Catwalk. Cells are joined by an edge if players were seen moving directly between them at least twice, or if they are horizontal neighbours whose floors differ by at most 48 units. Keeping the largest connected component leaves $N = 1,999$ cells and 6,780 edges. Each cell takes the most common game place name among the positions recorded in it, which gives the 23 callouts, and a representative standing point (the mean recorded position, moved onto a walked spot). Walking distances between cells are shortest paths on this graph. A finer 16×16 -unit raster records which spots were ever walked on, and at what heights, for the ray casting below.

5.3 Observation model: who would have seen whom

The core of the observation model is the probability $\pi_o(i)$ that friendly observer o would have an enemy standing in cell i on the radar. We model it with a logistic regression (Hastie et al., 2009) on binned features:

$$\text{logit } \pi_o(i) = \alpha_{s(o,i),d(o,i)} + \beta_{h(o,i)} + \gamma_{v(o,i)}, \quad (4)$$

where d is one of eight distance bins (edges at 250, 500, 750, 1,000, 1,500, 2,000 and 3,000 units), h and v are the horizontal and vertical angles between the observer's view direction and the target's chest (48 units above the floor) in 5-degree bins, and s is a sight-line class from two ray casts, precomputed for every pair of cells, between the eye points (64 units above the floor) of the observer's cell and the target's cell. The *walk* ray cast treats every raster pixel nobody ever stood on as a wall, and also blocks rays that pass below rising terrain or through the slab of an upper floor. It is precise but closes open areas that nobody walks, such as the air

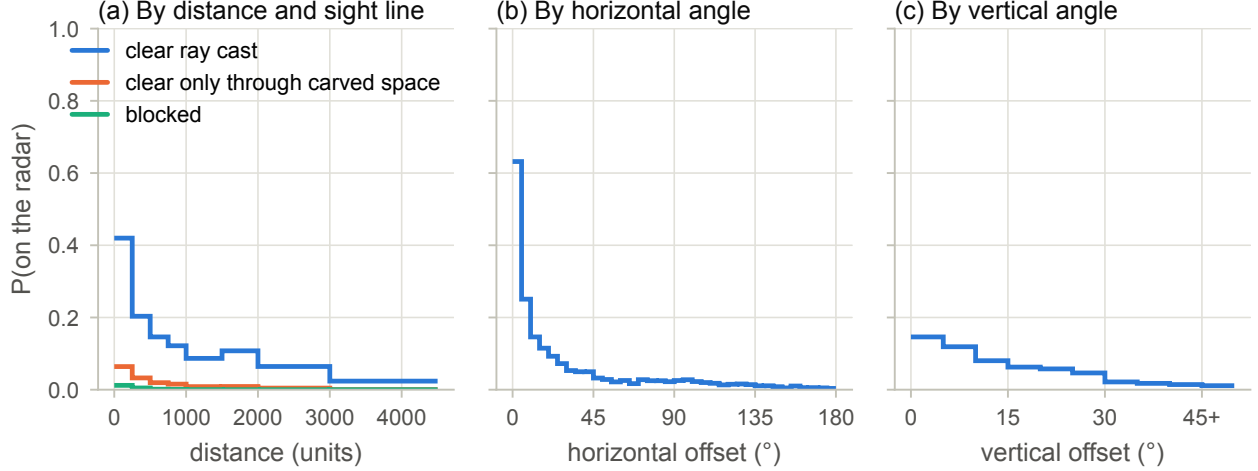


Figure 5: The fitted spotting model, as the probability that an enemy standing in a cell is on the radar at a sampled tick, before the per-pair correction. (a) By distance and sight-line class, for a target 10–15° off the view direction horizontally and within 5° vertically. (b) By horizontal offset, for a clear sight line at 500–750 units. (c) By vertical offset, same conditions as (a) at 500–750 units.

above a drop. The *carve* ray cast additionally opens every pixel through which a *real* spotting ray, from an observer’s eyes to the spotted enemy’s chest, passed at a similar height, since every recorded sighting proves its sight line was clear. Every walk-clear line is also carve-clear, so s takes three values: clear in both, clear only after carving, or blocked.

The coefficients are fitted by maximum likelihood with a small L_2 penalty, using L-BFGS (Byrd et al., 1995), on aggregated counts over about 27 million observer–enemy pairs from the training demos, excluding pairs hidden by a smoke or a flash. The fitted model separates spotted from unspotted pairs with a held-out area under the ROC curve of 0.99: given one pair where spotting happened and one where it did not, it ranks the former higher 99% of the time (Hanley and McNeil, 1982). The additive model cannot know about individual windows or boxes, so it is multiplied by a per-cell-pair correction, an empirical-Bayes ratio of observed to expected sightings shrunk towards 1 with 3 pseudo-counts (Efron and Morris, 1975). Pairs of cells that never appeared together therefore keep the geometric estimate. Finally, a sight line passing within 130 units of an active smoke is blocked, and $\pi_o(i)$ is capped at 0.9 so that no cell is ever treated as certainly empty.

Figure 5 shows the fitted terms. Spotting falls steeply with distance and with the angle off the view direction. The peak within 5 degrees probably reflects, in part, reverse causation: players turn their crosshair towards enemies they have seen.

Negative information. An alive enemy who is not on the radar is evidence against every cell the team could see. Assuming observers detect independently given the enemy’s cell,

$$L_t(i) = P(\text{enemy not on the radar} \mid x_t = i) = \prod_{o \in O_t} (1 - \pi_o(i)), \quad (5)$$

where O_t is the set of alive, unblinded friendly players. The update of Equation (2) multiplies the belief by L_t^γ ; a tempering exponent $\gamma < 1$ would soften the evidence, but $\gamma = 1$ was best on validation.

Kill feed. When an enemy kills a friendly player, the killer must have had a line of sight to where the victim died. We weight cell i by $K(i) = 0.05 + 0.95 \tilde{\pi}(i) / \max_j \tilde{\pi}(j)$, where $\tilde{\pi}(i)$ is the spotting probability between the victim’s cell and i with the view-angle terms at their best. For wallbangs and kills through smoke only

range is used: $K(i) = 1$ within 2,000 units and 0.05 beyond. The floor of 0.05 keeps a mistaken likelihood from eliminating the truth.

5.4 Motion models

Grid Markov chain (HMM). A first-order Markov chain over cells, one step per 0.25 s, is estimated by counting recorded transitions C_{ij} between consecutive steps of every alive player. The transition matrix is conditioned on the enemy's side ς (T or CT), the bomb phase ϕ , and a bin τ of the time since the player was last on the opponents' radar: 0–2, 2–5, 5–10, 10–20 or more than 20 seconds, or never this round. Players who were just seen tend to hold or fall back, while players unseen for a while rotate along routes. Rare contexts would give noisy estimates, so each level is shrunk towards the one above it:

$$T^{(\varsigma, \phi)} = \text{norm}(C^{(\varsigma, \phi)} + 0.5 S), \quad T^{(\varsigma, \phi, \tau)} = \text{norm}(C^{(\varsigma, \phi, \tau)} + 4 T^{(\varsigma, \phi)}), \quad (6)$$

where norm normalizes rows to sum to one and $S = \text{norm}(I + A)$ spreads pseudo-counts over each cell and its grid neighbours (A is the adjacency matrix). Adding λ times a stochastic matrix adds λ pseudo-observations to each row, distributed like the parent model. The prediction step is $\bar{b}_t = b_{t-1} T^{(\varsigma, \phi_t, \tau_t)}$.

Trajectory-library particles (PF). A first-order chain forgets a player's heading at every step, so its predictions spread diffusively, while real players run along routes: ten seconds after a sighting they can be 2,000 units away. Our second motion model therefore replays real trajectories. The *library* holds every recorded state of every alive player in the training rounds at 0.25 s spacing, 4,054,305 states in 18,197 trajectories. Each state stores its cell, round time, bomb phase, time since that player was last spotted, team economy, velocity, and a pointer to the next state of the same trajectory. Each enemy is tracked by $M = 512$ particles, each a pointer ℓ_m into the library with a weight w_m . At every step a particle simply advances to the next state of its recording, $\ell_m \leftarrow \text{next}(\ell_m)$.

When the enemy is sighted in cell j , all particles are redrawn from library states of players on the enemy's side in j (widening to up to three rings of neighbouring cells if fewer than 40 states are there). Candidate ℓ is drawn with probability proportional to a product of similarity kernels:

$$\kappa(\ell) = \exp\left(-\frac{(t_\ell - t)^2}{2 \cdot 15^2}\right) \cdot 0.02^{*[\phi_\ell \neq \phi_t]} \cdot \exp\left(-\frac{(\ln(1 + \tau_\ell) - \ln(1 + \tau))^2}{2 \cdot 0.5^2}\right) \cdot 0.7^{|b_{\text{buy}_\ell} - b_{\text{buy}}|} \cdot k_v(\ell), \quad (7)$$

with round time t in seconds and τ the time since last seen (the τ kernel is replaced by 0.05 when exactly one of the two states has never been seen). The heading kernel k_v compares the enemy's velocity v with the candidate's velocity v_ℓ . It is used only when the enemy was also seen within the previous 0.5 s, so that v can be measured; otherwise, as on a first sighting, $k_v = 1$. If both move faster than 80 units/s, it is a von Mises kernel $\exp(3(\cos \theta_\ell - 1))$ in the angle between them (Mardia and Jupp, 2000). It is 0.3 for a stationary candidate when the enemy moves, and $\exp(-|v_\ell|/15)$, with speeds in units per step, when the enemy stands still. On later steps the weights absorb the evidence: $w_m \leftarrow w_m L_t(\text{cell}(\ell_m)) K_t(\text{cell}(\ell_m))$, then renormalize. When the effective sample size $1/\sum_m w_m^2$ falls below $M/3$, the particles are resampled systematically. Duplicated particles are then re-matched to fresh recordings from their current state, so that copies do not share one future. A particle whose recording ends (the player died or the round ended) is re-matched the same way. The belief is the weighted histogram of particle cells, smoothed over two rings of neighbours ($0.4 \hat{b} + 0.4 \hat{b}P + 0.2 \hat{b}P^2$ with P the neighbour-averaging matrix), since 512 particles are a sparse sample of 1,999 cells.

The matching kernel deliberately omits some variables. Matching also on the numbers alive on each side, on the same player's own trajectories, or on the weapon last seen was tried and did not help (Section 7.10).

Baselines. LASTSEEN keeps all probability on the last sighting. PRIOR ignores all evidence and uses where that side’s players stand, by bomb phase and 5-second bin of round time, in the training data (Figure 4b); PRIOR+NEG adds negative information and the kill feed. DIFFUSE is Isla’s occupancy map: a lazy random walk (stay with the empirical stay probability, otherwise move to a uniformly chosen neighbour) with negative information and kills.

5.5 The ensemble

Particles give sharp, route-shaped beliefs but can miss a route none of them took. The grid chain and the prior are blunter but put some probability on every plausible cell. The ensemble mixes them:

$$b_t^{\text{ENS}} = 0.75 b_t^{\text{PF}} + 0.15 b_t^{\text{HMM}} + 0.10 b_t^{\text{PRIOR+NEG}}. \quad (8)$$

In importance-sampling terms this is a *defensive mixture* (Hesterberg, 1995): the blunt components bound how badly the ensemble can be surprised. The Markov-chain filters also mix in a uniform floor of 10^{-6} per step, and if an observation ever removes all probability, which means the models disagree with reality, the filter keeps its prediction.

5.6 Molotovs

Players avoid burning molotovs. We learn two radial profiles from the training demos, in 20-unit bins of the distance r from a fire’s centre on the same floor (Figure 6). The *occupancy* profile $O(r)$ is the density of players standing at distance r while a fire burns, relative to the same distances in an equally long window starting one second after it has burnt out. It is 5–15% of normal within 120 units, rises to half the usual density at 180 units, which we take as the fire radius, and is normal beyond 200. The *step* profile $m(r)$ multiplies the grid transitions,

$$P(i \rightarrow j) = \frac{T_{ij} m(r_j)}{\sum_k T_{ik} m(r_k)}, \quad (9)$$

so that a blocked move’s probability goes to the other moves from the same cell (waiting at the edge or going around). It is fitted by maximum likelihood on about 66,000 recorded moves that started next to a burning fire. The grid chain uses the step profile, and the ensemble’s *output* is multiplied by O and renormalized. It is applied to the output only, because a burning fire is one standing condition and must not count as a fresh observation at every step (Section 8).

5.7 Team coordination

The filters above track each enemy independently, so seeing three Terrorists on B says nothing about the other two. Real teams are strongly correlated. We correct each enemy’s callout belief with recorded whole-team configurations. The *team library* stores, every 2 s of every training round, the callouts of each side’s alive players together with the clock (round time before a plant, time since the plant after it), the bomb phase, the team’s economy and the number alive: 133,726 snapshots.

At step t , let k enemies be alive with callout beliefs P_e (one-hot for an enemy on the radar). Candidate snapshots q are those from the same side and bomb phase with at least k players alive and a clock within 3 s of the current one. Each gets a context weight

$$\omega_q = \exp\left(-\frac{(\kappa_q - \kappa_t)^2}{2 \cdot 2^2}\right) \cdot 0.7^{|buy_q - buy_t|} \cdot 0.5^{n_q - k}, \quad (10)$$

where κ is the clock and n_q the snapshot’s number of players. Let π be the candidates’ weighted callout occupancy. Each enemy’s evidence enters as a likelihood ratio $r_e(k') \propto (P_e(k')/\pi(k'))^\beta$ with $\beta = 1$,

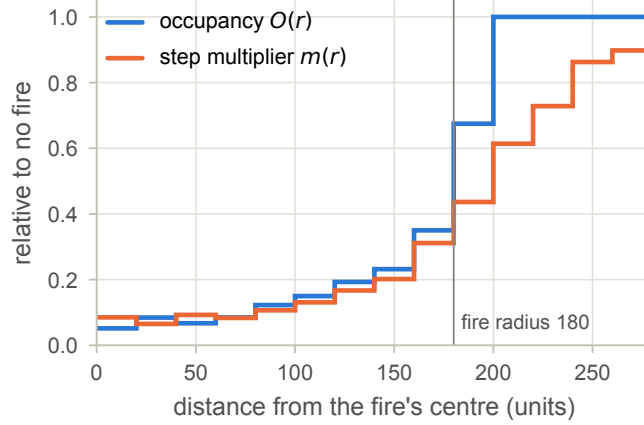


Figure 6: Molotov profiles fitted on the training demos, relative to the same distances without a fire. The occupancy $O(r)$ weights the output belief; the step multiplier $m(r)$ reshapes the grid transitions. Both are 1 beyond 280 units.

normalized so that $\sum_{k'} \pi(k') r_e(k') = 1$. A snapshot's players are unlabelled, so we sum over every *pairing* s , an injective map from our k enemies to the n_q recorded players (at most $5!/0! = 120$ pairings):

$$W_{q,s} = \frac{\omega_q}{|\Sigma_{k,n_q}|} \prod_e r_e(c_{q,s(e)}), \quad Q_e(k') = \frac{\sum_{q,s} W_{q,s} \mathbb{1}[c_{q,s(e)} = k']}{\sum_{q,s} W_{q,s}}, \quad (11)$$

where $c_{q,j}$ is the callout of recorded player j and $\Sigma_{k,n}$ is the set of pairings. Q_e is enemy e 's callout distribution when the recorded teams are reweighted by how well they explain *all* enemies' evidence at once. If recorded teammates were independent, the same computation would give $Q_e^{\text{ind}} = \pi \cdot r_e \propto P_e$. We therefore apply only the ratio $R_e = \tilde{Q}_e / Q_e^{\text{ind}}$, which is exactly 1 when recorded teams carry no information about correlations, and rescale each cell's belief by $R_e(c(i))$. To avoid over-reacting when few recorded teams match, Q_e is shrunk towards the independent answer by credibility weighting (Bühlmann, 1967): $\tilde{Q}_e = (n_{\text{eff}} Q_e + \lambda Q_e^{\text{ind}}) / (n_{\text{eff}} + \lambda)$, where $n_{\text{eff}} = (\sum_q W_q)^2 / \sum_q W_q^2$ is the effective number of recorded teams and $\lambda = 100$.

Summing over pairings marginalizes over the unknown correspondence between our enemies and the recorded players, much like joint probabilistic data association in multi-target tracking (Fortmann et al., 1983). The sum is a matrix permanent, which is hard to compute in general (Valiant, 1979) but tiny here. The alternative of keeping only each snapshot's best pairing, which is what Hungarian matching would choose (Kuhn, 1955), was clearly worse on validation. The correction is applied only once some enemy has been seen this round (Section 8 explains why).

5.8 Calibration by time since last seen

Finally, the callout probabilities are passed through a monotone map fitted on the validation maps. For enemies last seen τ seconds ago, with τ in group g (0–5 s, 5–20 s or 20 s and longer),

$$\tilde{q}(k) = \frac{\sigma(a_g \logit q(k) + b_g)}{\sum_{k'} \sigma(a_g \logit q(k') + b_g)}, \quad (12)$$

where σ is the logistic function. This is Platt scaling (Platt, 2000) applied to every callout and renormalized. It is increasing in q , so it never changes which callout ranks first, only how confident the model is. For small q , $\sigma(a \logit q + b) \approx e^b q^a$, so a_g acts as an inverse temperature: $a > 1$ sharpens the distribution and $a < 1$ softens it. Each cell keeps its share of its callout, $b(i) \leftarrow b(i) \tilde{q}(c(i)) / q(c(i))$. Enemies not yet seen this

Algorithm 1 One step of the ensemble filter for a hidden enemy e . Lines 12–14 reshape only the reported belief.

Require: particles $\{(\ell_m, w_m)\}_{m=1}^{512}$, grid beliefs h_{t-1} (HMM) and p_{t-1} (PRIOR+NEG), information $\mathcal{I}_t$

- 1: $\ell_m \leftarrow \text{next}(\ell_m)$ for all m ; re-match particles whose recording ended ▷ predict
- 2: $\bar{h}_t \leftarrow h_{t-1} T^{(\mathcal{S}, \phi_t, \tau_t)}$, with fire-blocked moves redirected; $\bar{p}_t \leftarrow$ occupancy prior at round time t
- 3: $w_m \leftarrow w_m L_t(\text{cell}(\ell_m)) K_t(\text{cell}(\ell_m))$; $h_t \propto \bar{h}_t \odot L_t \odot K_t$; $p_t \propto \bar{p}_t \odot L_t \odot K_t$ ▷ update
- 4: **if** e is on the radar in cell j **then**
- 5: redraw every ℓ_m from library states near j with probability $\propto \kappa(\ell)$ (Equation (7))
- 6: $w_m \leftarrow 1/512$ for all m ; $h_t \leftarrow \delta_j$
- 7: **else if** $1/\sum_m w_m^2 < 512/3$ **then**
- 8: systematic resampling; duplicates re-matched to fresh recordings
- 9: **end if**
- 10: $h_t \leftarrow (1 - 10^{-6}) h_t + 10^{-6}/N$
- 11: $b_t \leftarrow 0.75 \text{smooth}(\text{hist}(\ell, w)) + 0.15 h_t + 0.10 p_t$
- 12: $b_t \leftarrow$ team correction with R_e (Equation (11)), once any enemy has been seen ▷ output only
- 13: $b_t \leftarrow \text{norm}(b_t \odot O_t)$ while molotovs burn
- 14: $b_t \leftarrow$ calibration of callout shares (Equation (12)) by the group of τ_t^e
- 15: **return** b_t

round are left unchanged. The map family and the grouping were chosen by 4-fold cross-validation over whole validation rounds (Section 7.5). The fitted parameters are $(a_g, b_g) = (1.57, 1.59)$ for 0–5 s, $(1.04, 0.50)$ for 5–20 s and $(0.80, -0.10)$ beyond 20 s: the raw filter is too unsure right after a sighting and too sure long after it.

6 Experimental Setup

Models. We compare the full model (ENS) with the baselines of Section 5.4, with each motion model alone (PF, HMM), and with named ablations: ENS_0 , the ensemble before the four late enhancements (molotovs, team coordination, calibration, and the tested but disabled weapon matching); versions without negative information (PF–neg, HMM–neg) or without the kill feed; and a copy of ENS_0 with a different random seed, which measures Monte Carlo noise.

Evaluation moments. A *sample* is a (step, enemy) pair where the enemy is alive, off the radar, and was seen earlier in the round. Every round is scored from both teams’ points of view, since each side is “friendly” once. The 16 HLTV test maps yield 102,486 samples in 315 rounds with at least one sample.

Metrics. Let $k^\star$ be the callout and $i^\star$ the cell where the enemy really is.

- **Callout top-1 / top-3:** whether $k^\star$ is the most likely callout, or among the three most likely; ties count against the model. Random guessing gets 4.3% top-1.
- **Callout log-loss:** $-\ln q(k^\star)$, the average surprise at the truth in nats. It is minimized in expectation only by honest probabilities. A uniform guess over callouts scores $\ln 23 = 3.14$. The *cell log-loss* $-\ln b(i^\star)$ is the same over cells (uniform: $\ln 1,999 = 7.60$).
- **Expected calibration error (ECE):** every callout probability of every sample is binned (edges 0, 0.05, 0.1, 0.2, ..., 0.9, 1), and ECE is the bin-size-weighted mean of |mean stated probability – observed frequency|.

Table 2: Held-out professional test set: 16 HLTV maps, 102,486 samples of enemies seen earlier in the round and now off the radar. Round-bootstrap 95% intervals in parentheses. Higher is better for accuracy, lower for log-loss.

Model	Callout top-1	Callout top-3	Callout log-loss	Cell log-loss
ENS (full model)	0.481 (0.456–0.508)	0.727 (0.701–0.752)	1.665 (1.581–1.747)	5.822 (5.685–5.951)
ENS ₀ (before enhancements)	0.479 (0.454–0.505)	0.726 (0.701–0.751)	1.689 (1.600–1.777)	5.849 (5.706–5.982)
PF (trajectory particles)	0.472 (0.447–0.497)	0.718 (0.694–0.742)	1.875 (1.745–2.016)	6.917 (6.631–7.200)
HMM (grid Markov chain)	0.466 (0.438–0.495)	0.704 (0.674–0.736)	2.591 (2.347–2.838)	7.060 (6.753–7.370)
DIFFUSE (random walk)	0.444 (0.417–0.471)	0.689 (0.660–0.720)	3.023 (2.743–3.312)	7.850 (7.513–8.191)
LASTSEEN	0.422 (0.394–0.452)	0.584 (0.555–0.613)	7.196 (6.834–7.529)	16.671 (16.599–16.738)
PRIOR+NEG	0.215 (0.193–0.236)	0.447 (0.423–0.470)	2.609 (2.552–2.672)	7.259 (7.160–7.363)
PRIOR	0.213 (0.192–0.235)	0.451 (0.427–0.475)	2.607 (2.550–2.670)	7.257 (7.157–7.361)
Uniform	–	–	3.135	7.600

- **Team-level count error:** at each moment, the expected number of hidden enemies in each callout (the sum of their callout probabilities) is compared with the actual count, $\sum_k (\sum_e q^e(k) - n_k)^2$. This is a proper score for the expected counts.

Two readings of log-loss help build intuition. First, e^{LL} is the number of equally likely callouts the forecaster is effectively undecided between (its *perplexity*). Second, log-loss has a betting interpretation. Suppose a bookmaker offers fair odds $1/q_B(k)$ on each callout according to model B , and a bettor using model A spreads their bankroll over the callouts in proportion to q_A , the Kelly strategy for mutually exclusive outcomes. Their bankroll is multiplied by $q_A(k^*)/q_B(k^*)$ on each bet, so its expected logarithmic growth rate is exactly $\text{LL}_B - \text{LL}_A$ nats per bet (Kelly, 1956; Cover and Thomas, 2006).

Uncertainty. Consecutive moments of a round are nearly the same situation, so the 102,486 samples are far from independent. All 95% intervals come from a cluster bootstrap that resamples whole rounds, with 2,000 replicates (Field and Welsh, 2007). Both viewpoints of a round share a cluster. Model differences are *paired*: in every replicate both models are scored on the same resampled rounds, so difficulty they share cancels, and a difference can be clearly nonzero even when the two models’ separate intervals overlap. Particle filters are random, so we also compare two runs that differ only in their random seed; a claimed gain smaller than that difference is not a gain.

Protocol. All settings (kernel widths, mixture weights, shrinkage strengths, calibration family) were chosen on validation data: the 8 HLTV validation maps, or the FACEIT validation matches during early development. The test maps were scored during development to track progress, but never used to choose a setting. All test results in Section 7 come from single evaluation runs of the final code and data.

7 Results

7.1 Main results

Table 2 gives the headline comparison. The full model is the best in every metric. Against the last-seen heuristic, it names the right callout 6.0 points more often (paired 95% interval 4.0 to 7.9) and has the right callout in its top three 14.2 points more often (12.0 to 16.4). Against the population prior the gains are 26.8 and 27.5 points. Its callout log-loss of 1.665 corresponds to a perplexity of $e^{1.665} \approx 5.3$ callouts, against 13.6 for the prior and 23 for a uniform guess. In the betting reading, a bankroll wagered according to ENS against a bookmaker pricing with PRIOR grows by a factor of $e^{0.94} \approx 2.6$ per bet on average (in the geometric sense). LASTSEEN has respectable accuracy but a catastrophic log-loss: it puts all its probability on one callout and is

Table 3: What matters: paired differences on the test set (first model minus second), with round-bootstrap 95% intervals. Positive $\Delta\text{top-1}$ and negative $\Delta\text{log-loss}$ favour the first model. HMM is the learned grid chain, PF the replayed trajectories and DIFFUSE the untrained random walk; “-neg” removes negative information.

Comparison	Δ callout top-1	Δ callout log-loss	Δ cell log-loss
<i>Learned motion</i>			
HMM vs. DIFFUSE	+0.022 (+0.005, +0.040)	-0.432 (-0.558, -0.308)	-0.791 (-0.958, -0.630)
PF vs. DIFFUSE	+0.027 (+0.009, +0.045)	-1.148 (-1.367, -0.941)	-0.933 (-1.161, -0.707)
PF vs. HMM	+0.005 (-0.012, +0.022)	-0.716 (-0.893, -0.549)	-0.143 (-0.327, +0.050)
<i>Negative information</i>			
PF vs. PF-neg	+0.021 (+0.007, +0.036)	-0.044 (-0.080, -0.007)	-0.094 (-0.186, -0.016)
HMM vs. HMM-neg	+0.015 (+0.006, +0.024)	-0.191 (-0.261, -0.129)	-0.245 (-0.317, -0.179)
<i>Ensemble and evidence</i>			
ENS ₀ vs. PF (the mixture)	+0.007 (+0.002, +0.013)	-0.186 (-0.261, -0.127)	-1.068 (-1.238, -0.913)
ENS vs. no kill feed	+0.001 (-0.002, +0.004)	-0.006 (-0.011, -0.002)	-0.006 (-0.018, +0.007)
<i>Monte Carlo noise</i>			
ENS ₀ vs. another seed	+0.000 (-0.002, +0.003)	-0.002 (-0.010, +0.007)	-0.001 (-0.020, +0.019)

wrong 58% of the time, so a Kelly bettor using it goes broke. Adding negative information and the kill feed to the prior changes almost nothing (PRIOR+NEG vs. PRIOR), because the prior does not know where the enemy was last seen.

7.2 What matters

Table 3 isolates the components. *Learned motion* is the largest single ingredient. Replaying recorded trajectories instead of a random walk cuts the callout log-loss by 1.15 nats, and it beats the learned grid chain by 0.72 nats while matching its accuracy. The chain’s probabilities are badly spread: it often ranks the right callout first, but it assigns too little probability to where fast-moving players actually go. *Negative information* improves every motion model, adding 2.1 points of accuracy to the particles and 1.5 to the grid chain. The *mixture* of particles with the grid chain and the prior adds another 0.7 points and removes 0.19 nats of log-loss, and more than a nat of cell log-loss, by insuring against routes no particle took. The kill feed gives a small but reliable log-loss gain; it matters because 48% of killers are not on the radar at the moment of the kill. The seed comparison bounds Monte Carlo noise at about ± 0.003 in accuracy and ± 0.01 in callout log-loss on this test set. Validation ablations, not repeated on test, showed that conditioning the motion on the time since the last sighting was the largest single fix, roughly halving the cases where the true position received almost no probability, and that knowing the correct bombsite after the plant also helps.

7.3 Accuracy as the trail goes cold

Figure 7 breaks the results down by the time since the last sighting (full table in Section B). In the first two seconds most enemies have not left their callout, so ENS and LASTSEEN are nearly tied in accuracy (88.9% vs. 88.0%), but the model’s log-loss is six times lower (0.31 vs. 1.82) because it hedges the remaining cases. The gap then widens: 42.4% against 37.2% at 10–20 s, and 25.2% against 14.5% after 40 s. By 20–40 s, LASTSEEN is barely better than the population prior (23.0% vs. 20.3%), and after 40 s it is worse (14.5% vs. 21.1%), while the model stays above both. Its log-loss rises smoothly towards the prior’s as the information ages, never exceeding it. This is the behaviour one wants from a forecaster that knows what it does not know.

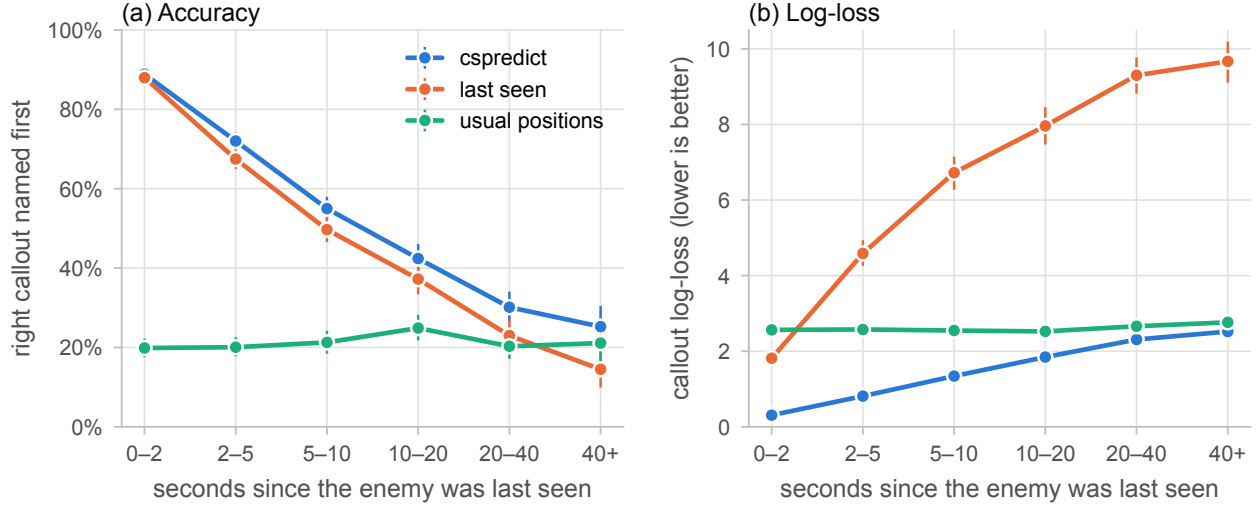


Figure 7: Test-set accuracy and log-loss by time since the enemy was last seen, with round-bootstrap 95% intervals. “Usual positions” is PRIOR+NEG. Each bin holds 11,256 to 23,229 samples.

Table 4: Callout top-1 accuracy when one friendly player is left (test set), by the number of enemies alive. Callout log-loss of ENS in the last row.

	1v1	1v2	1v3	1v4	1v5
Samples	2,001	4,292	5,963	8,701	3,179
ENS	0.597	0.509	0.476	0.400	0.411
ENS ₀	0.592	0.515	0.485	0.387	0.408
LASTSEEN	0.425	0.421	0.376	0.343	0.394
PRIOR+NEG	0.361	0.349	0.304	0.157	0.131
ENS callout log-loss	1.324	1.632	1.608	2.027	1.836

7.4 Clutch situations

Rounds often end with one player left against several (a “clutch”), and these are the moments where a coach most wants to know what the lone player could have inferred. Table 4 shows callout accuracy from the lone player’s side. The model’s lead over LASTSEEN is largest in the 1-versus-1 (59.7% vs. 42.5%), where a single opponent’s movement dominates. Each slice contains only 40 to 98 rounds, so differences of a point or two between ENS and ENS₀ are within noise. Against four or five opponents, the full model’s log-loss is clearly better than ENS₀’s (−0.041 and −0.038 nats).

7.5 Calibration

Table 5 shows how the calibration map was chosen. A single map for all moments barely helps (at best −0.005 nats), because the raw model errs in opposite directions at different horizons. Fitting separate maps by the time since the last sighting gains about four times as much (−0.020 to −0.022, depending on the family). Figure 8 shows why on the test set. Within 5 s of a sighting the raw ensemble is *under*-confident (above the diagonal), because its 10% population-prior component caps how sure it can be. Across all horizons, its most confident bin claimed 92% on average and was right 98.5% of the time. After 20 s it is *over*-confident: stated probabilities averaging 44% and 54% came true only 36% and 47% of the time. On the test set the calibrated model’s calibration error falls from 0.0040 to 0.0012 overall, and in every horizon group (Figure 8). Table 6 lists the overall reliability. Calibration left top-1 accuracy unchanged by construction and improved

Table 5: Choosing the calibration map: 4-fold cross-validated callout log-loss of ENS before calibration, on the 8 validation maps (63,906 samples, 166 rounds; folds are whole rounds). “By horizon” fits separate parameters for 0–5 s, 5–20 s and 20 s and longer since the last sighting.

Map family	One map for all moments	By horizon
None	1.6873	–
Power (temperature scaling), 1 parameter	1.6856	1.6669
Platt (logit-affine), 2 parameters	1.6845	1.6651
Piecewise linear in $\log q$, 4 parameters	1.6821	1.6673

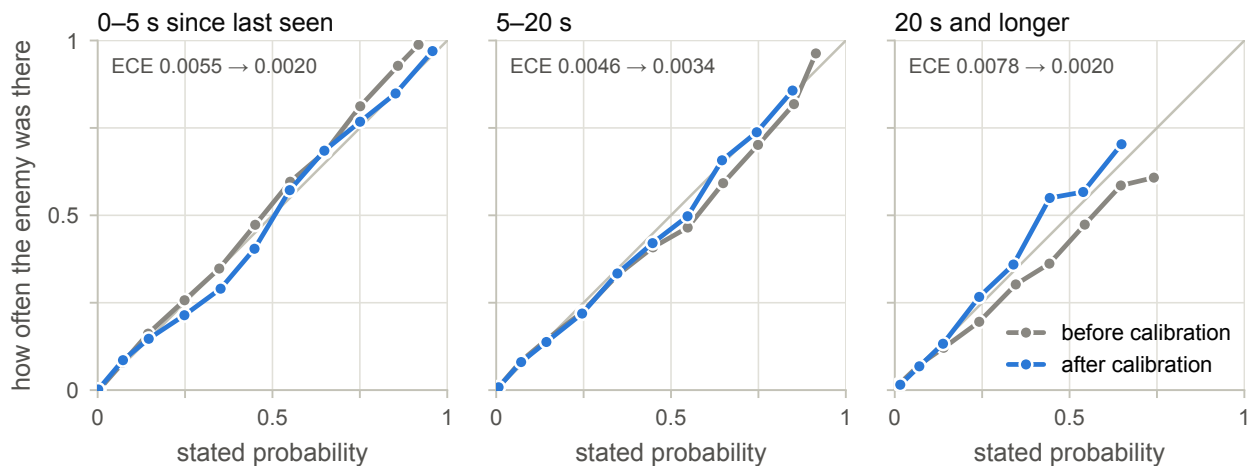


Figure 8: Reliability of the callout probabilities on the test set, by time since the enemy was last seen, before and after calibration. A perfectly calibrated forecaster lies on the diagonal. Every callout probability of every sample is binned; bins with fewer than 200 probabilities are omitted. ECE: expected calibration error.

the callout log-loss by 0.021 nats (paired interval 0.010 to 0.032).

7.6 The four enhancements

The ensemble ENS_0 was extended in four steps, each tested on validation first. Table 7 reports each step’s paired effect on test relative to the step before, together with the subset of moments where the step is designed to act. Overall, ENS improves on ENS_0 by 0.024 nats of callout log-loss (0.012 to 0.036) and 0.027 of cell log-loss (0.015 to 0.040); its top-1 gain of +0.002 (−0.001 to +0.005) is within noise. Most of the gain is concentrated where each step should act: molotovs help while a fire burns (19% of samples), and the team correction helps enemies nobody has seen yet once one of their teammates has been. In the betting reading, 0.024 nats is a bankroll growing by about 2.4% per bet against the earlier model.

7.7 Coordination and the team as a whole

The test set contains 246,193 samples of enemies who have not been seen yet in the round while at least one of their teammates has. For these, the team correction raises top-1 accuracy from 0.175 to 0.191 and lowers callout log-loss from 2.607 to 2.576. That brings the model level with PRIOR+NEG (0.191, 2.600) on enemies it has no direct evidence about, which the independent filters could not reach. At the team level, the expected number of hidden enemies per callout misses the actual count by a squared error of 2.908 per moment after first contact (2.787 to 3.036), against 2.931 for the independent filters and 3.264 for the prior. The model still underestimates stacks: where it expects 1.5–2 enemies in one callout there are 1.95 on average, and where it expects 2–3 (mean 2.26) there are 2.48.

Table 6: When the model states a callout probability in a given range, how often the enemy was really there (test set, all horizons).

Stated	20–30%	30–40%	40–50%	50–60%	60–70%	70–80%	80–90%	90%+
Before calibration	21.2%	32.0%	40.4%	50.0%	62.3%	74.6%	89.0%	98.5%
After calibration	24.0%	33.4%	43.6%	52.9%	67.0%	75.2%	85.2%	97.0%

Table 7: Paired effect of each enhancement on the test set, relative to the step before (95% round-bootstrap intervals), and where it shows.

Step	Δ top-1	Δ callout LL	Δ cell LL	Where it shows
Weapon at last sighting (tested, left off)	−0.001 (−0.005, +0.003)	−0.006 (−0.014, +0.002)	−0.012 (−0.029, +0.007)	Nowhere beyond Monte Carlo noise
Molotovs	+0.001 (−0.000, +0.002)	−0.001 (−0.002, +0.000)	−0.004 (−0.007, −0.002)	While a fire burns: top-1 +0.004 (−0.000, +0.009), cell LL −0.017 (−0.027, −0.006)
Team coordination	+0.001 (−0.002, +0.005)	−0.002 (−0.007, +0.003)	−0.002 (−0.007, +0.003)	Never-seen enemies after first contact: top-1 +0.016 (+0.011, +0.022), LL −0.031 (−0.040, −0.022); count error −0.021 (−0.034, −0.007)
Calibration	0 (by design)	−0.021 (−0.032, −0.010)	−0.021 (−0.032, −0.010)	Reliability (Figure 8)

Two design details mattered on validation. Summing over all pairings (Equation (11)) beat keeping each snapshot’s single best pairing by a wide margin. And applying the correction before first contact *hurt*. With no sightings, every enemy’s belief is the same, so the product in Equation (11) multiplies five copies of one likelihood ratio and counts the shared negative evidence five times.

7.8 Training data: more versus purer

Table 8 compares models trained on the FACEIT matches, the professional matches, or both, all scored on the professional test set before calibration so that they are compared alike. Pooling the 35 FACEIT matches with the 47 professional ones beats the professional matches alone by 1.1 points of accuracy (0.1 to 2.1) and 0.037 nats of log-loss (0.018 to 0.057). For this model, more data beats purer data.

7.9 The FACEIT development set

On the 6 FACEIT test matches (48,876 samples), with models trained and calibrated on FACEIT data only, ENS reaches 0.463 top-1 (0.429–0.498) and 1.724 callout log-loss (1.606–1.841), against 0.378 and 7.834 for LASTSEEN and 0.219 and 2.603 for PRIOR. The four enhancements improve log-loss by 0.062 nats (0.042 to 0.082), more than on professional matches. Professional play is more predictable. LASTSEEN alone is right 42% of the time on professional matches but only 38% on FACEIT, because professionals hold positions more, and in 1-versus-1 clutches ENS names the right callout 60% of the time on professional matches but only 25% on FACEIT.

7.10 Features that did not help

We report these because each was a plausible hypothesis, and several are features a knowledgeable player would suggest first.

Table 8: Effect of the training data on the professional test set (ENS before calibration).

Trained on	Callout top-1	Callout log-loss	Cell log-loss
FACEIT only (35 maps)	0.463	1.767	6.163
HLTV only (47 maps)	0.470	1.723	5.939
Both (82 maps)	0.481	1.686	5.843

- **The weapon in hand at the last sighting.** The signal is real: AWP snipers move less in the 5 s after being seen (median 190 vs. 300 units for riflers), and players seen holding a knife or grenade move more. Yet matching on the weapon, whether on every weapon class, on guns only, or on AWP versus the rest, never beat Monte Carlo noise on validation or on test (Table 7). Over 80% of sightings are rifles or pistols, where the weapon mostly repeats what position and economy already say.
- **Man advantage and player identity.** Matching trajectories on the numbers alive on each side (1v3 plays differently from 5v5), or preferring the same player’s own recordings, did not help on the FACEIT validation data. Even soft versions left too few matching trajectories. Both are implemented but disabled, and they have not been re-tested on professional data.
- **Molotovs inside the particle filter.** Making particles wait at a fire’s edge failed, because real players often re-route, so waiting particles fell out of step with their recordings. Re-weighting particles by the occupancy profile at every step counted one standing fact as a new observation at every step. Both were slightly worse, within noise. Only the grid-level and output-level versions are used.
- **Flashed teammates.** A flashed player really does stop spotting, from about a quarter of the flash duration until 1.25 times it. However, only one mid- or late-round moment in eight has a teammate flashed now or in the previous 5 s, and rebuilding the flashes missing from 55 HLTV demos moved validation log-loss by only -0.001 (-0.006 to $+0.005$). The gain is confined to the 5 s after a flash, where cell log-loss improves by 0.029 (0.003 to 0.062).
- **Smoke timing.** Fixing the parsing bug that cut 15% of smokes 4 s short initially made results slightly worse. It helped only once smokes were also treated as see-through for their last 1.5 s, which matches when spotting through them resumes in the data.

7.11 Comparison with a general-purpose model

Can a general-purpose model do the same job given the same information? We queried TypeSafe’s Jev (`jev-1.13.0`; TypeSafe, 2026), a hosted model that answers multiple-choice questions posed as text with a probability for every option. For each hidden enemy it was asked which of the 23 callouts the enemy is in. It received a structured description of only what the team knew, and it never saw later events, true positions or our model’s beliefs. Facts computed in code were labelled as such: travel times from the last sighting, spotting coverage per callout, and base rates from the *training* rounds. Every variant was developed on 2,000 validation moments; then all variants were run once on 3,000 test moments. Calibration, blend weights and a simple baseline formula were fitted on validation.

Table 9 shows the results. Given a plain description, the model mostly repeats the last sighting (84% of its top picks), its accuracy is no better than LASTSEEN, and its raw probabilities are over-confident (log-loss 4.07). A two-parameter calibration per horizon group halves that log-loss. Following the vendor’s own advice (focused per-enemy questions, facts computed in code, and splitting “which callout?” into “still there?” and “if not, where?”) improved its probabilities step by step. Supplying movement rates from the training rounds (of the players last seen in that callout that long ago, the share still there and where the others went) finally broke the anchoring: its last-seen rate fell to 40%, close to the true 42%, and its top-3 accuracy rose by 9.8 points (7.8 to 11.6) over the plain variant. Its top-1 accuracy still only ties LASTSEEN (-0.003 ; -0.035 to

Table 9: A general-purpose model on 3,000 held-out professional moments (a random subset of the test samples). The last column is how often the top pick is the callout where the enemy was last seen; the enemy really was still there in 42% of these moments.

Model	Callout top-1	Top-3	Log-loss	Picks last-seen callout
CSPREDICT (ENS)	0.488 (0.457–0.520)	0.737	1.648	59%
Formula: 9 weights on the same facts, no Jev	0.457 (0.425–0.490)	0.710	1.819	75%
Jev, split questions + movement rates, calibrated	0.416 (0.387–0.449)	0.703	1.944	40%
Jev, split questions, calibrated	0.416	0.683	2.023	75%
Jev, focused per-enemy questions, calibrated	0.419	0.642	2.038	86%
Jev, plain description + travel times, calibrated	0.404	0.629	2.182	76%
Jev, plain description, calibrated	0.406 (0.375–0.443)	0.605	2.180	84%
Jev, plain description, raw	0.406	0.605	4.066	84%
LASTSEEN	0.420 (0.388–0.455)	0.584	7.150	100%
PRIOR	0.233	0.466	2.589	19%

+0.030), and a 9-parameter formula on the same facts beats it by 4.1 points (1.1 to 6.9). Blending 20% of Jev into that formula improves log-loss by 0.034 (0.020 to 0.049), so the model extracts something the formula misses, but not much. All experiments together cost about US\$2.60 in API calls.

8 Discussion

Why replayed trajectories beat a Markov chain. A first-order chain’s next position depends only on its current cell, so it cannot represent momentum: a player sprinting down a corridor is modelled as equally likely to turn back at every step. The belief therefore spreads like a diffusing cloud, growing roughly as the square root of time, while real players move *ballistically* along routes until they stop, covering distance in proportion to time. Replaying recorded trajectories preserves momentum, route choice and typical stopping points, and it represents the branching into distinct routes directly. The price is sampling noise and the risk that no particle took the route the enemy chose, which the defensive mixture with the grid chain and the prior covers (Table 3). This mirrors the classical trade-off between analog and model-based forecasting (Lorenz, 1969).

The power and the danger of negative information. Absence of evidence is the most abundant evidence in this problem, and treating it properly improves every model. But Equation (5) is applied every 0.25 s under an independence assumption. If the spotting model wrongly believes a cell is visible, the error compounds across steps, and the filter can become confident that the enemy is not somewhere it is. The 0.9 cap on π_o , the uniform floor and the mixture components all limit this. One plausible contributor to the late-horizon over-confidence of Figure 8 is exactly this accumulation of correlated negative evidence; the calibration corrects the symptom, not the cause.

Counting evidence once. Two of our design lessons are instances of one principle. Bayes’ rule assumes that each observation is new. Re-weighting particles by a burning fire at every step treats one standing fact as dozens of independent observations. Applying the team correction before first contact multiplies five identical copies of the same evidence. In both cases the fix was to apply the information exactly once: fire occupancy only on the output, and the team correction only after sightings make the enemies’ beliefs distinct.

Why calibration must depend on the horizon. The raw ensemble’s miscalibration has two sources that dominate at opposite ends. Just after a sighting, the probability held by the blunt mixture components, above all the 10% population prior, makes the model under-confident. Long after it, compounded negative evidence

and particle degeneracy make it over-confident. A single global map must compromise between the two, which is why it helped so little (Table 5), while horizon-specific maps fixed both.

When a real signal adds nothing. The weapon result is a reminder that a variable can be correlated with the outcome and still add no information beyond what the model already conditions on. Here, position and economy largely determine which weapon a player carries and how they play. Man-advantage matching failed for a different reason: every extra matching variable divides the pool of similar recorded trajectories, and with a few dozen training maps (35 FACEIT maps when this was tested) the pool runs dry, the familiar curse of dimensionality (Bellman, 1961).

Structure versus generality. The general-purpose model anchored on the most salient cue, the last sighting, until it was handed movement statistics that the structured model learns directly. Even then, a nine-parameter formula on the same facts did better. With a few dozen matches of training data, building the structure of the problem into the model (motion, sight lines, Bayes’ rule) is far more data-efficient than asking a general model to infer it.

9 Limitations and Ethical Considerations

Scope and data. Everything is learned per map, and we have studied one map with 71 professional and 46 FACEIT matches. The 16 test maps give intervals of about ± 2.5 points on top-1 accuracy. Teams recur across the train/test split, so the results measure generalization to new matches, not to unseen teams. Most professional teams in the data are tier two or three.

Information left out. Sound (footsteps, gunshots, reloads) is not used, by choice. It is probably the largest source of information a real team has that the model ignores, and including it is the most promising extension. We also assume enemies are identifiable on the radar and that molotovs are common knowledge.

Modelling approximations. Coordination enters as a correction on top of independent filters rather than as a joint model, and the model still underestimates how many enemies stack in one callout. The geometry is learned from where players stand, so windows and rarely walked spots are approximate; exact geometry would improve the spotting model. Blinding is modelled from 0 to 1 times the flash duration, whereas the data suggest a window from about 0.25 to 1.25 times; that refinement is untested.

Fair play. A live overlay built on this model would show players information beyond what the game gives them. Competitive platforms prohibit third-party software that gives an unfair advantage (FACEIT, 2026), and Hladky and Bulitko (2008) noted that such an overlay would be nearly undetectable, which makes the rule more important, not less. We built and evaluated CSPREDICT strictly for post-match review, coaching and analysis. Uses such as broadcast graphics, bot behaviour, or anti-cheat research, which could flag players who act on information they should not have, are compatible with that intent.

10 Conclusion

We have shown that the reasoning players and coaches do about unseen opponents in Counter-Strike 2 can be made quantitative, calibrated and testable. A Bayes filter built entirely from recorded matches, with a learned map, a learned spotting model, and particles that replay recorded trajectories, names the right callout for a hidden opponent 48% of the time, with probabilities that mean what they say. The gains come mostly

from two ideas that are old in filtering but new to this setting: negative information and analog motion. Coordination between opponents and horizon-dependent calibration add smaller, targeted improvements, and several intuitive features add nothing. Natural next steps are sound, more maps and seasons, joint models of whole teams, and learned motion models such as those of [Durst et al. \(2024\)](#) inside the same probabilistic framework.

Acknowledgments

We thank the authors of the X-Ego-CS dataset for releasing their recordings under the MIT license, HLTV.org for hosting professional demos, and the maintainers of demoparser2 and awpy. The software and this manuscript were developed with the assistance of an AI coding assistant (Claude, Anthropic).

References

- Richard Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961.
- Curt Bererton. State estimation for game AI using particle filters. In *AAAI Workshop on Challenges in Game Artificial Intelligence*, 2004.
- Christopher Berner et al. Dota 2 with large scale deep reinforcement learning. arXiv:1912.06680, 2019.
- Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1): 1–3, 1950.
- Hans Bühlmann. Experience rating and credibility. *ASTIN Bulletin*, 4(3):199–207, 1967.
- Richard H. Byrd, Peihuang Lu, Jorge Nocedal, and Ciyu Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on Scientific Computing*, 16(5):1190–1208, 1995.
- Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2nd edition, 2006.
- Morris H. DeGroot and Stephen E. Fienberg. The comparison and evaluation of forecasters. *Journal of the Royal Statistical Society: Series D (The Statistician)*, 32(1–2):12–22, 1983.
- Arnaud Doucet, Nando de Freitas, and Neil Gordon, editors. *Sequential Monte Carlo Methods in Practice*. Springer, 2001.
- David Durst, Feng Xie, Vishnu Sarukkai, Brennan Shacklett, Iuri Frosio, Chen Tessler, Joohwan Kim, Carly Taylor, Gilbert Bernstein, Sanjiban Choudhury, Pat Hanrahan, and Kayvon Fatahalian. Learning to move like professional Counter-Strike players. *Computer Graphics Forum*, 43(8):e15173, 2024. Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation.
- Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- Bradley Efron and Carl Morris. Data analysis using Stein’s estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

- FACEIT. What is deemed to be a cheat? <https://support.faceit.com/hc/en-us/articles/360015788779-What-is-deemed-to-be-a-cheat>, 2026. Accessed September 2026.
- Christopher A. Field and Alan H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B*, 69(3):369–390, 2007.
- Thomas E. Fortmann, Yaakov Bar-Shalom, and Molly Scheffe. Sonar tracking of multiple targets using joint probabilistic data association. *IEEE Journal of Oceanic Engineering*, 8(3):173–184, 1983.
- Tilman Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.
- I. J. Good. Rational decisions. *Journal of the Royal Statistical Society: Series B*, 14(1):107–114, 1952.
- Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. Novel approach to nonlinear/non-Gaussian Bayesian state estimation. *IEE Proceedings F (Radar and Signal Processing)*, 140(2):107–113, 1993.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.
- James A. Hanley and Barbara J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2nd edition, 2009.
- Tim Hesterberg. Weighted average importance sampling and defensive mixture distributions. *Technometrics*, 37(2):185–194, 1995.
- Stephen Hladky. Predicting opponent locations in first-person shooter video games. Master’s thesis, University of Alberta, 2009.
- Stephen Hladky and Vadim Bulitko. An evaluation of models for predicting opponent positions in first-person shooter video games. In *Proceedings of the IEEE Symposium on Computational Intelligence and Games (CIG)*, pages 39–46, 2008.
- HlTV.org. Counter-strike match results and GOTV demos. <https://www.hltv.org>, 2026. Demos downloaded September 2026.
- Damián Isla. Probabilistic target tracking and search using occupancy maps. In Steve Rabin, editor, *AI Game Programming Wisdom 3*. Charles River Media, 2006.
- Yonghyun Jeong, Hyunjin Choi, Byoungjip Kim, and Youngjune Gwon. DefogGAN: Predicting hidden information in the StarCraft fog of war with generative adversarial nets. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 4296–4303, 2020.
- Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- John L. Kelly, Jr. A new interpretation of information rate. *Bell System Technical Journal*, 35(4):917–926, 1956.
- Genshiro Kitagawa. Monte Carlo filter and smoother for non-Gaussian nonlinear state space models. *Journal of Computational and Graphical Statistics*, 5(1):1–25, 1996.

- Augustine Kong, Jun S. Liu, and Wing Hung Wong. Sequential imputations and Bayesian missing data problems. *Journal of the American Statistical Association*, 89(425):278–288, 1994.
- Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- LaihoE. demoparser2: A Counter-Strike 2 demo parser. <https://github.com/LaihoE/demoparser>, 2026. Software; accessed September 2026.
- Edward N. Lorenz. Atmospheric predictability as revealed by naturally occurring analogues. *Journal of the Atmospheric Sciences*, 26(4):636–646, 1969.
- Kanti V. Mardia and Peter E. Jupp. *Directional Statistics*. Wiley, 2000.
- Mahdi Pakdaman Naeni, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, pages 2901–2907, 2015.
- John C. Platt. Probabilities for SV machines. In Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large-Margin Classifiers*, pages 61–74. MIT Press, 2000.
- Lawrence D. Stone. *Theory of Optimal Search*. Academic Press, 1975.
- Gabriel Synnaeve, Zeming Lin, Jonas Gehring, Dan Gant, Vegard Mella, Vasil Khalidov, Nicolas Carion, and Nicolas Usunier. Forward modeling for partial observation strategy games – a StarCraft defogger. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- Bulent Tastan, Yuan Chang, and Gita Sukthankar. Learning to intercept opponents in first person shooter games. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, pages 100–107, 2012.
- Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.
- TypeSafe. Jev documentation. <https://docs.typesafe.ai/introduction>, 2026. Model jev-1.13.0; accessed September 2026.
- Leslie G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, 1979.
- Yunzhe Wang, Soham Hans, and Volkan Ustun. X-Ego: Acquiring team-level tactical situational awareness via cross-egocentric contrastive video representation learning. arXiv:2510.19150, 2025.
- Ben G. Weber, Michael Mateas, and Arnav Jhala. A particle model for state estimation in real-time strategy games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 7, pages 103–108, 2011.
- Peter Xenopoulos and Claudio Silva. ESTA: An esports trajectory and action dataset. arXiv:2209.09861, 2022.
- Peter Xenopoulos, Harish Doraiswamy, and Claudio Silva. Valuing player actions in Counter-Strike: Global Offensive. In *Proceedings of the IEEE International Conference on Big Data*, pages 1283–1292, 2020.
- Peter Xenopoulos et al. awpy: Counter-Strike demo parsing and analytics in Python. <https://github.com/pnxenopoulos/awpy>, 2026. Software, version 2; accessed September 2026.
- Georgios N. Yannakakis and Julian Togelius. *Artificial Intelligence and Games*. Springer, 2018.

A Hyperparameters

Table 10: Constants of the final model. Unless noted, values were set once and checked on validation.

Component	Setting
Sampling	64-tick demos stored at 8 Hz; filter step 0.25 s (16 ticks)
Grid	64-unit columns; floors split at a 110-unit height gap; a floor needs 8 samples (1 s); step edges up to 48 units; 16-unit raster for ray casting
Spotting model	8 distance bins; 36 horizontal and 10 vertical 5° bins; 3 sight-line classes; L_2 penalty $0.005\ w\ ^2$; rays sampled every 8 units up to 4,500; eye height 64, chest 48; smoke radius 130; pair correction with 3 pseudo-counts, clipped to $[0.02, 50]$; $\pi_o \leq 0.9$
Information state	smoke blocks from +1.0 s after burst to -1.5 s before expiry; molotov burns from +0.5 s; blind if flash > 0.3 s; economy edges 1,500 and 3,800
Economy estimate	released code only (Section 3); 11 situations with 3 pseudo-rounds at the overall frequencies; gun likelihoods with add-one smoothing, exponent 0.7 (fitted on training rounds)
Negative information	$\gamma = 1$
Kill feed	$K = 0.05 + 0.95 \bar{\pi} / \max \bar{\pi}$; wallbang or smoke: 1 within 2,000 units, else 0.05
Grid chain	time-since-seen bins 0–2, 2–5, 5–10, 10–20, 20+ s and never; smoothing 0.5; shrinkage 4 pseudo-counts
Particles	$M = 512$; ≥ 40 candidates (widen up to 3 rings); round-time kernel $\sigma = 15$ s; other bomb phase $\times 0.02$; time-since-seen kernel $\sigma = 0.5$ in $\ln(1 + \tau)$, seen/never mismatch $\times 0.05$; economy $\times 0.7$ per level; heading von Mises $\kappa = 3$ above 80 units/s; resample when $ESS < M/3$; smoothing weights 0.4 / 0.4 / 0.2
Ensemble	0.75 particles, 0.15 grid chain, 0.10 prior with negative information; uniform floor 10^{-6}
Prior	5-s round-time bins by side and bomb phase; 0.01 pseudo-count per cell
Molotovs	20-unit bins to 280; same floor within 100 units; control window from 1 s after burnout; step profile bounded to $[0.01, 1]$
Team correction	snapshots every 2 s; clock $\sigma = 2$ s, window ± 3 s; economy $\times 0.7$ per level; $\times 0.5$ per extra recorded player; $\beta = 1$; probability floor 10^{-3} ; shrinkage $\lambda = 100$; at most 5,000 candidates; only after first contact
Calibration	Platt map per horizon group (0–5, 5–20, 20+ s): $(a, b) = (1.57, 1.59), (1.04, 0.50), (0.80, -0.10)$

B Additional Results

Table 11: Test-set callout top-1 and callout log-loss by time since the last sighting (the data of Figure 7).

		0–2 s	2–5 s	5–10 s	10–20 s	20–40 s	40 s+
	Samples	11,256	14,004	16,612	21,292	23,229	16,093
Top-1	ENS	0.889	0.720	0.550	0.424	0.301	0.252
	ENS ₀	0.889	0.715	0.544	0.427	0.298	0.250
	LASTSEEN	0.880	0.675	0.497	0.372	0.230	0.145
	PRIOR+NEG	0.199	0.201	0.213	0.249	0.203	0.211
Log-loss	ENS	0.308	0.812	1.342	1.845	2.310	2.520
	ENS ₀	0.365	0.831	1.358	1.852	2.351	2.532
	LASTSEEN	1.815	4.587	6.723	7.962	9.300	9.669
	PRIOR+NEG	2.565	2.576	2.548	2.525	2.661	2.765

B.1 Improper scores favour over-confidence

Table 12 scores the same test samples with two intuitive but improper measures: the probability placed within 300 walking units of the true position, and the expected walking distance from the belief to the truth. By both, LASTSEEN looks better than the full model. Both measures reward concentrating probability on a single point, whether or not that concentration is justified. A forecaster optimizing them would report over-confident beliefs. This is why the paper’s conclusions rest on log-loss and accuracy.

Table 12: Improper scores on the test set (higher is better for mass within 300 units, lower for expected distance). They rank the models differently from the proper log-loss.

Model	Mass within 300 units	Expected distance (units)	Callout log-loss (proper)
ENS	0.274	1,099	1.665
PF	0.287	1,022	1.875
HMM	0.319	916	2.591
DIFFUSE	0.300	902	3.023
LASTSEEN	0.339	855	7.196
PRIOR	0.045	1,923	2.607

B.2 Team-level counts

Table 13: Expected against actual numbers of hidden enemies in a callout, after first contact (test set, ENS), binned by the expected number. The model is accurate for sparse callouts and underestimates stacks.

Model expects	0–0.1	0.1–0.25	0.25–0.5	0.5–0.75	0.75–1	1–1.5	1.5–2
Mean expected	0.029	0.161	0.347	0.613	0.857	1.173	1.691
Mean actual	0.030	0.159	0.338	0.585	0.849	1.313	1.952

C Reproducibility

All results can be regenerated from the demo files with the project’s command-line tools (Python 3.12):

```
python -m cspredict.parse # demos -> parquet tables
python -m cspredict.build --sources hltv xego # grid, spotting, motion, fires, teams
python -m cspredict.evaluate --train-sources hltv xego --sources hltv \
--split val --models ens_uncal --out outputs/final/val_pooled_uncal
python -m cspredict.calibrate --train-sources hltv xego \
--run outputs/final/val_pooled_uncal --model ens_uncal
python -m cspredict.evaluate --train-sources hltv xego --sources hltv --split test \
--models last_seen prior prior_neg diffuse hmm_noneg hmm pf_noneg pf \
ens ens_nokill ens_uncal ens_indep ens_nofire ens_w0.5 ens_seed1 \
--ref ens_nofire --focus last_seen prior_neg ens_nofire ens --out outputs/final/test_pooled
python paper/make_figures.py # figures and the paired comparisons
```

The training-data comparison (Table 8) and the FACEIT results repeat the build, calibrate and evaluate steps with `-sources hltv` or `-sources xego`; the Jev benchmark is `python -m cspredict.jev_lab` (see docs/jev.md). Model names in the code map to the paper as follows: `ens` is ENS, `ens_nofire` is ENS₀, `pf` and `hmm` are PF and HMM, `diffuse`, `last_seen`, `prior` and `prior_neg` are the baselines, and the suffix `_noneg` removes negative information.

The numbers here come from git commit 66a5aeb, which took the opponents’ economy class from their true equipment values. Later code estimates it (Section 3); there, `ens_truebuy` reproduces the uncalibrated ENS exactly.